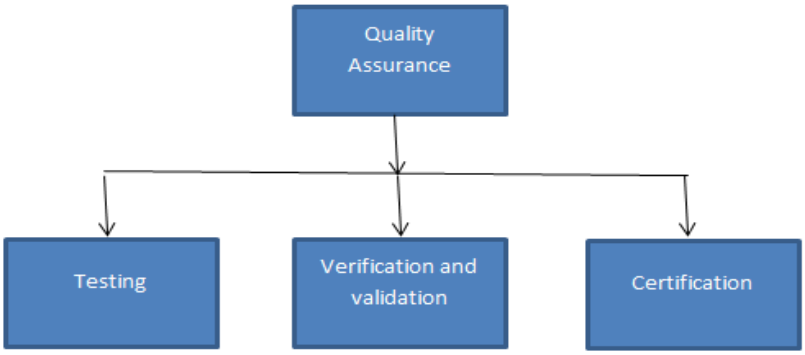
Unit-4

System quality assurance: -

- System quality assurance is very important in system development.
- Quality assurance is the review of system and related documentation for completeness, correctness, reliability and maintainability.
- It also includes the assurance that the system meets the specifications and the requirements for its intended use and performance.
- Quality assurance is an **umbrella** activity which must be applied throughout the system development process.
- Quality assurance defines the objectives of the system and reviews the overall activities so that if there are errors they are corrected early in the development process.
- In each phase, various actions are taken to ensure that there are no errors in the final system and assure that various standards established are followed throughout the process.

Levels of system quality assurance

Various levels of quality assurance are shown below:



1. Testing

The success of any project depends on its ability to perform accurately and according to the specifications, specified before the start of the project.

Hence, testing of the system is performed to find the errors and correct them.

The main objectives of testing are:

- To systematically uncover different types of errors in minimum time and with a minimum amount of effort.
- To ensure that the system is reliable and working as per the specifications.
- To ensure that the system is according to the requirements of the users.

2. Verification and Validation

- Verification and Validation is the checking process that ensures that the system conforms to its specifications and meets the requirements.
- The system should be verified and validated at each stage of the system development process.

3. Certification

- Certification is the seal of approval given for the correctness of the system.
- To certify the system, the agency appoints a team of specialists who carefully examine the documentation for the system to determine whether the system does what the vendor claims to do. This is done by testing the system and comparing the results with what the vendor had said about his system.

Quality assurance goals

Every stage in the system development life cycle has the goal of quality assurance. The goals and

their relevance to the quality assurance of the system are:

1. Quality Factors Specifications

The goal of this phase is to define the quality factors. System quality factors are:

(i) Correctness

Correctness is the extent to which:

- System is free from design defects and from coding defects, i.e., fault-free.
- System meets its specified requirements.
- System meets user expectations.

(ii) Reliability

Reliability is the ability of a program to perform a required function under stated condition for a stated period of time.

(iii) Efficiency

Efficiency is the extent to which system performs its intended functions with a minimum consumption of computing resources.

(iv) Integrity:- Integrity is the extent to which access to system and data is denied to unauthorized users.

(v) Portability:- Portability is the ease with which system can be transferred from one computer system or environment to another.

(vi) Accuracy:- Accuracy is a qualitative assessment of freedom from errors.

(vii) Robustness:- Robustness is the extent to which system can continue to operate correctly despite the introduction of invalid inputs.

(viii) Testability:- Testability is the effort required to test the programs for their functionality.

(ix) Maintainability:- Maintainability is the effort required to locate and fix an error in a program.

(x) Usability:- Usability is the effort required to learn, operate, prepare input and interpret output of a program.

2. Software Requirements Specification (SRS)

The quality assurance goal of this stage is to generate the required document that contains the concise and clear specification of the functional, performance, design and interface requirements of the proposed system.

The SRS document should have the following main characteristics:

I. Completeness:- SRS should be complete. It should fully describe functionality to be

delivered. It should contain all types of input and should provide features for handling all functions of the system.

II. Accurate:- SRS should be accurate. It must accurately describe functionality to be built.

Every requirement must be a true requirement of the system.

III. Unambiguous:- SRS is unambiguous if every stated requirement has only one interpretation. SRS should not be confusing.

IV. Consistent:- SRS should be consistent. It is consistent if and only if no subset of

individual requirements described in it is conflict.

V. Modifiable:- SRS should be modifiable. It should be changeable as the requirements

change while maintaining its completeness and consistency.

VI. Verifiable:- A requirement is verifiable if and only if there exists some cost-effective process that can check whether the final product meets the requirements.

VII. Valid:- SRS should be valid. A valid SRS is that where all parties related to the project can understand and accept it.

3. Software Design Specifications

- The software design document defines the overall architecture of the software including the functions and features as per the software requirements document.
- It also defines the logical subsystem and physical modules to ensure that all conditions are covered.

4. Software Testing and Implementation

- To ensure the completeness and accuracy of the system and minimize the retesting process is the goal of the testing phase.
- In the implementation phase, the goal is to provide a logical order for creation of modules and thus the creation of the system.

5. Maintenance and Support

- To continue to work as per the original specifications this phase defines the necessary software adjustment plan.
- The quality assurance goal is to review the overall activities so that if there are errors they are corrected at the earliest and it ensures that there are no errors in the final system and various standards established are followed throughout the process.

Importance of Quality Assurance in System Design

 It is important to consider quality challenges in the start of the development.

 Quality starts at the designing phase and its presence dominates throughout the development process.

 Quality needs to be designed into the product.

 Quality assurance eliminates as many defects and challenges as possible from design and installations.

 The most useful thing that quality assurance can do during the design phase is to make sure that the supplied specifications have a set of clear, testable goals.

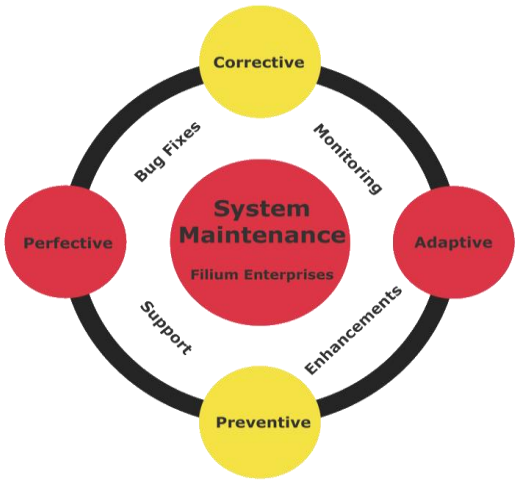
 A lack of quality in the design process can invalidate good requirement specification and can make correct implementation impossible.

 A good design is a testable design. It is important to consider during design - how to achieve goals and how to evaluate whether the goals are achieved or not.

 Quality assurance needs to be planned to start the test automation right from the start of the development. This will impact the quality assurance efforts in the project.

System Maintenance

- After completion of implementation phase, the system is required to be properly maintained. System maintenance is an activity that occurs following delivery of a system product to the customer.



- It primary is goal to modify and update software application after delivery to correct error and improve performance.
- It is last phases of software development life cyclic
- System maintenance is an important activity in the life of a system. The total cost of maintenance phase is much higher than the development cost of the system. Usually it consumes about 40-70% of the cost of the entire life cycle.

Importance of studying the maintenance of the system

Maintenance activities involve:

- Understanding the existing system.
- Making enhancements to system.
- Understanding the effect of change.
- Adapting products to new environments.
- Making the changes to both code and documents.
- Correcting problems.
- Testing the new parts &
- Retesting the old parts that were not changed.

Aims of software maintenance

Aims of software maintenance are:

- To maintain the value/quality of software overtime.
- To enhance the software product by providing new functional capabilities.
- To adapt the software to cope with changes in the environment. It involves moving the software to different machines; modify it to accommodate new protocol etc.
- Modification and revalidation of software to correct errors.
- Upgrading the performance characteristics of a system.
- To upgrade the software in anticipation of future problems.
- To improve user displays and modes of interaction.

Types of system maintenance

Three types of system maintenance are:

1. Corrective Maintenance
2. Preventive maintenance
3. Adaptive Maintenance
4. Perfective Maintenance

1. Corrective Maintenance

- Corrective maintenance refers to modifications initiated by defects in the system software.
- Thus the main aim of corrective maintenance is to correct any remaining errors in the system software.
- A problem can result from specification errors, design errors, logic errors, coding errors, testing errors etc.
- Defects are also caused by data processing errors and system performance errors.

2.Preventive maintenance

- Preventive maintenance refers to the proactive maintenance activities carried out on equipment, machinery, or systems to prevent unexpected failures and ensure their continued functioning at optimal levels.

3. Adaptive Maintenance

- Adaptive maintenance means modifying the software product to react to changes in the outside environment in which the system operates.
- The term environment refers to the totality of all conditions and influences that act from outside upon the software.
- For example, porting to a new compiler & operating system, business rules, government policies,
- Adaptive maintenance is generally not requested by the client but it is imposed by the outside environment.
- For example, if the government increases DA then software system of Payroll will have to undergo adaptive changes.

4. Perfective Maintenance

Perfective maintenance means improving the following aspects of the software product.

- Efficiency
- Performance
- Maintainability
- Effectiveness

System Documentation

- System documentation refers to the collection of documents that provide detailed information about a software system, its components, functionalities, and usage.
- Documentation is the way of representing in written all the information of details of the system.

- It serves as a comprehensive reference for various stakeholders involved in the development, deployment, maintenance, and operation of the system.
- It is the process of preparing, organizing, storing and maintaining historical records and other documents used during the various phases of the life cycle of the system.
- A good documentation is a sign of the professional pride. It increases understandability and maintainability of a system.
- Good documentation is an aid to - Analysis, communication, debugging, evaluation and training to personnel.

Need for Documentation

Documentation is needed to serve the following purposes:

- To depict what the system is supposed to be and how it should perform its functions.
- To review the programs or development of the system.
- To communicate facts about the system to users.
- To provide operating instructions.
- To illustrate both technically and economically how a system would better serve the objectives and goals of the organization.
- To provide necessary guidelines to allow correction or revision of a system.
- To assist in the reconstruction of the system.

Types of Documentation

- **System Requirements Document**: This document outlines the functional and non-functional requirements of the system, including user needs, system capabilities, and constraints.
- **Architecture Documentation:**
  - Describes the high-level structure of the system, including its components, interactions, and dependencies.
  - It may include diagrams such as system architecture diagrams, data flow diagrams, and component diagrams.
- **User Manual/User Guide:** Provides guidance for users on how to use the system, including instructions, tutorials, and troubleshooting tips.
- **Testing Documentation**: Describes the testing approach, test cases, and test results for the system, including unit tests, integration tests, and acceptance tests.

- **Maintenance Documentation**: Offers guidance for maintaining the system, including information about updates, patches, backups, and troubleshooting common issues.
- **Security Documentation**: Describes the security measures implemented in the system to protect against threats and vulnerabilities, including access controls, encryption, and auditing mechanisms.
- **Training Materials**: If necessary, documentation or materials for training users or administrators on how to use or manage the system effectively

Need of standard software documentation

- Documentation standards are the guidelines relating to the structure of documentation, content and its other features.
- Standards must be established for all documentation to be produced during the software production Documentation standards in a software project are important.

They are needed for every type of documentation because of the following reasons:

1. To aid understandability

There are a number of documentation styles, norms, tools and methods. Some may be useful, some may be redundant with other documentation or some others may be too complicated to understand. Therefore the methods which have been proved as samples of best practice or the methods that make the documentation understandable to all developers and readers should be adopted.

2. To save money

A high proportion of software process costs is incurred in producing documentation. So there is a need to avoid over documentation to save customer money. There is a need to have standard approach to documentation, leaving unnecessary documentation out and concentrating on the important aspects of the project.

3. To maintain document quality

Document quality is as important as software quality, Documentation standards act as basis for documentation quality assurance

4. To save time

A considerable portion of software development effort is absorbed by documentation that costs considerable amount of developers' time. Standard documentation provides a framework for recording essential information needed throughout the development.

5. To transfer projects

To modify previously developed software or to transfer or extend the project, the new designers or developers without the extensive background of the project can also benefit from standard documentations.

6. To aid clarity

The goal of software documentation standard is to provide a clear description of the software management, engineering, assurance processes and products. It makes clear the system architecture and workflows etc.

Documentation Standards

There are three types of documentation standards:

1. Documentation Process Standards

- Documentation Process Standards define the process that is to be followed to produce the documents.
- These record the procedures involved in document development, the software tools used for document production and also checking and refinement procedures to ensure high quality documents
- Process Standards define the approach to be taken in producing documents.

2. Document Standards

- Document standards are applicable to all documents produced during a software development.
- These standards govern the structure and presentation of documents.

3. Document Interchange Standards

- These standards are needed as electronic copies of documents are interchanged.
- The use of interchange standards allows documents to be transferred electronically and re-created in their original form.
- It refers to the usage of common version of a system for document production. For example, the use of a standard style sheet.

System evaluation

- System evaluation involves assessing the performance, effectiveness, and efficiency of various components within a system, such as hardware, software, processes, and procedures.

- The purpose of system evaluation is to identify strengths, weaknesses, opportunities, and threats as well as areas for improvement

System evaluation step:

- **Define Objectives:**
  - Clearly define the goals and objectives of the system evaluation.
  - Determine what aspects of the system will be evaluated and what criteria will be used to measure performance.
- **Gather Data:**
  - Collect relevant data and information about the system, including performance metrics, user feedback, error logs, and any other relevant sources of information.
- **Analysis:**
  - Analyze the collected data to assess the current state of the system.
  - Identify trends, patterns, and areas of concern.
  - Compare the system's performance against predefined benchmarks or industry standards.
- **Identify Strengths and Weaknesses:**
  - Determine the strengths and weaknesses of the system based on the analysis. This may include aspects such as reliability, scalability, security, usability, and performance.
- **Identify Opportunities for Improvement:**
  - Identify opportunities for enhancing the system's performance, efficiency, and effectiveness.
  - This may involve implementing new technologies, optimizing processes, or updating software and hardware components.
- **Risk Assessment:**
  - Evaluate potential risks and vulnerabilities associated with the system, such as security threats, data breaches, or system failures.
  - Develop strategies to mitigate these risks and improve the system's resilience.
- **Recommendations:**
  - Based on the findings of the evaluation, provide recommendations for improvement.
  - Prioritize recommendations based on their potential impact and feasibility of implementation.
- **Implementation Plan:**

- Develop a plan for implementing the recommended changes and improvements.
- Define specific actions, timelines, responsibilities, and resources required for implementation.

- **Monitoring and Review:**
  - Continuously monitor the system's performance after implementing the recommended changes.
  - Regularly review and evaluate the effectiveness of the improvements and make further adjustments as needed.